

# Prompt to Animated Video Module: Implementation Architecture

---

## 1. Objective

Add a production-ready Prompt to Animated Video capability to Vistarak using your prior VoiceFrame pipeline, while fitting the current architecture:

- Django backend as feature gateway
- External FastAPI microservices for heavy AI workloads
- Next.js frontend module pages

This plan is implementation-focused and aligned to your existing code organization.

## 2. Current Baseline (Observed in Repo)

- Backend is Django with feature apps and URL grouping under module routes.
- Existing features call external model services over HTTP (for example STT/TTS/translation pattern).
- Frontend talks directly to Django endpoints from module pages.
- Long-running tasks are currently mostly synchronous, but animated video generation requires async orchestration.

Conclusion: video generation should be introduced as a dedicated microservice with job queue semantics and polled status.

## 3. Target Architecture

### 3.1 Logical Flow

1. User submits prompt from frontend.
2. Django creates a job record and forwards request to Video Service.
3. Video Service enqueues async pipeline job.
4. Pipeline stages execute:
  - Script generation (Gemini)

- Scene assets (InvokeAI)
- Voice synthesis (Piper or existing TTS integration)
- Timeline assembly (MoviePy/FFmpeg)
- Final artifact upload (S3/MinIO)

5. Frontend polls Django for status and receives final downloadable/streamable URL.

## 3.2 Components

- Frontend page: new Animated Video UI (prompt form, style controls, progress timeline, preview/download).
- Django gateway layer:
  - Request validation
  - Job persistence and ownership
  - Proxy to Video Service endpoints
  - Normalized response format
- Video Service (FastAPI):
  - API endpoints for submit/status/cancel/result
  - Celery worker pipeline for long jobs
  - Redis broker/result backend
  - Artifact manager
- GPU model dependency:
  - InvokeAI as colocated container/service with configurable base URL
- Storage:
  - S3/MinIO for final video and optional intermediate manifests

## 4. Recommended Integration in This Repo

### 4.1 Backend (Django)

Create a dedicated Django app for video generation routing and persistence.

Suggested app name: `animated_video`

Responsibilities:

- Expose stable endpoints to frontend:
  - `POST /modules/animated-video/jobs/`
  - `GET /modules/animated-video/jobs/{job_id}/`
  - `POST /modules/animated-video/jobs/{job_id}/cancel/`
  - `GET /modules/animated-video/jobs/{job_id}/result/`

- Store job metadata and status locally (authoritative for product behavior).
- Forward processing requests to Video Service.
- Map service-specific states into product states:
  - `queued`, `script_generation`, `asset_generation`, `voice_generation`, `compositing`, `uploading`, `completed`, `failed`, `cancelled`

## 4.2 New Data Model (Django)

Create model `AnimatedVideoJob` with fields:

- `job_id` (uuid, primary key)
- `user_ref` (string for now, future FK)
- `prompt` (text)
- `style` (char)
- `target_duration_sec` (int)
- `status` (char)
- `progress` (int 0-100)
- `error_code` (char nullable)
- `error_message` (text nullable)
- `video_url` (text nullable)
- `service_job_id` (char nullable)
- `created_at`, `updated_at`, `completed_at`

Optional observability fields:

- `pipeline_trace_id`
- `cost_estimate_usd`
- `latency_ms`

## 4.3 Frontend (Next.js)

Add a new module page (or repurpose existing placeholder) with:

- Prompt input (required)
- Voice/style selectors (optional defaults)
- Duration hint (15 to 180 sec)
- Submit button
- Polling status panel (every 2-4 sec)
- Result panel (video player + download)

- Error panel with retry

Frontend should only call Django endpoints, not Video Service directly.

## 5. Video Service Design

### 5.1 Service API Contract

Base URL: configurable via env from Django.

Endpoints:

- `POST /api/v1/jobs`
  - Input: prompt, style, duration hint, voice profile, callback metadata(optional)
  - Output: service\_job\_id, queued timestamp
- `GET /api/v1/jobs/{service_job_id}`
  - Output: status, stage, progress, eta\_sec, error, artifact metadata
- `POST /api/v1/jobs/{service_job_id}/cancel`
  - Output: cancellation accepted/rejected

### 5.2 Async Execution

Use Celery + Redis inside the video service stack.

Why here (not in Django):

- Keeps heavy compute orchestration out of the monolith.
- Allows independent scaling of workers.
- Avoids blocking Django request threads.

### 5.3 Pipeline Stages

1. Validate and sanitize prompt.
2. Build structured scene plan using prompt template manager (Gemini).
3. Generate scene visuals via InvokeAI.
4. Generate dialogue audio via Piper/TTS.
5. Compose timeline using MoviePy/FFmpeg.
6. Upload final MP4 and return signed/public URL.
7. Emit status updates after each stage.

## 6. Prompt Architecture (Critical)

Because your ask is prompt-driven animated video generation, prompt design must be a first-class module.

### 6.1 Prompt Template Manager

Implement versioned templates in Video Service:

- `system_prompt_v1` : enforce JSON schema, educational tone, safe content
- `user_prompt_enricher` : append style, audience level, duration constraints
- `scene_constraints_prompt` : cap number of scenes and dialogue per scene

All generated scripts must be schema-validated before downstream steps.

### 6.2 Prompt Schema (Suggested)

Required output schema from LLM:

- `title`
- `estimated_total_duration`
- `scenes[]`
  - `scene_id`
  - `start_sec`, `end_sec`
  - `visual_description`
  - `camera_direction`
  - `dialogues[]` with `speaker`, `text`, `emotion`, `voice_gender`
  - `on_screen_text[]`

Reject and retry generation if schema invalid.

### 6.3 Safety and Quality Guardrails

- Input moderation before LLM call.
- Output moderation on script text.
- Hard limits:
  - max prompt length
  - max scenes
  - max dialogue chars per scene

- Deterministic fallback template if LLM fails repeatedly.

## 7. Config and Secrets

Add env-driven configuration for both Django and Video Service.

Django variables:

- `ANIMATED_VIDEO_SERVICE_URL`
- `ANIMATED_VIDEO_TIMEOUT_SEC`
- `ANIMATED_VIDEO_POLL_TIMEOUT_SEC`

Video Service variables:

- `GEMINI_API_KEY`
- `INVOKEAI_URL`
- `CELERY_BROKER_URL`
- `CELERY_RESULT_BACKEND`
- `VOICE_MODELS_PATH`
- `ARTIFACT_STORAGE_PROVIDER`
- `S3_BUCKET` , `S3_REGION` , credentials

## 8. Deployment Topology

### 8.1 Local Development

- Django backend
- Next.js frontend
- Video Service API
- Celery worker
- Redis
- InvokeAI
- Optional MinIO for local artifact storage

### 8.2 Production

- Keep Django stateless and CPU-focused.
- Deploy video workers on GPU nodes.
- Run Redis as managed or persistent service.

- Use object storage for final artifacts.
- Add API gateway timeout and retry policy for submit/status endpoints.

## 9. Observability

Minimum required telemetry:

- Per-job structured logs with correlation ID
- Stage-level latency metrics
- Failure rate per stage
- External API cost counters (Gemini/inference)
- Queue depth and worker utilization

Add health endpoints:

- `/health/live`
- `/health/ready`
- `/health/dependencies` (Redis, InvokeAI, storage)

## 10. Security

- Authenticate all Django endpoints (at least session/token guard).
- Validate MIME/file types for user uploads if any reference audio/images are added later.
- Signed URLs for artifact access.
- No secrets in code; only environment/secret manager.

## 11. Phased Implementation Plan

### Phase 0: Contract and Skeleton (2-3 days)

- Define API contracts between Django and Video Service.
- Create Django app with job model and endpoint stubs.
- Create frontend page shell and status polling hook.

Exit criteria:

- Frontend can create a mocked job and poll a mocked status.

### Phase 1: Prompt + Script Engine (3-4 days)

- Build prompt template manager and schema validator.
- Integrate Gemini script generation in Video Service.
- Persist intermediate scene manifest.

Exit criteria:

- Prompt reliably produces schema-valid scene JSON.

## Phase 2: Asset and Voice Generation (4–6 days)

- Integrate InvokeAI with retry/backoff.
- Integrate Piper/voice generation with caching.
- Add partial failure handling and retries per stage.

Exit criteria:

- Scene assets and dialogue audio generated for full scripts.

## Phase 3: Composition + Storage (3–4 days)

- Implement timeline compositor.
- Upload final MP4 to object storage.
- Return stable result URL via Django endpoint.

Exit criteria:

- End-to-end prompt to playable video URL.

## Phase 4: Hardening (3–5 days)

- Add observability, throttling, auth checks, and error taxonomy.
- Load test concurrent jobs.
- Optimize worker concurrency and GPU usage.

Exit criteria:

- Service-level objectives met for target workload.

# 12. Testing Strategy

Test layers:



- Unit tests: prompt schema validation, state mapping, payload builders
- Integration tests: Django to Video Service contract
- Pipeline tests: service with mocked Gemini/InvokeAI/TTS
- E2E tests: frontend submission to final video retrieval
- Chaos tests: dependency outage (InvokeAI down, Redis restart, storage failure)

## 13. Risks and Mitigations

- Long generation latency
  - Mitigation: async queue, progress stages, ETA reporting
- GPU bottlenecks
  - Mitigation: separate worker pools, autoscaling by queue depth
- LLM schema drift
  - Mitigation: strict JSON validation + retry + fallback template
- Cost growth
  - Mitigation: token limits, scene caps, per-job budgeting metrics

## 14. Immediate Next Actions in This Repo

1. Create Django app `animated_video` and register URLs under `/modules/animated-video/`.
2. Add `AnimatedVideoJob` model + migration.
3. Implement Django endpoints as proxy/adaptor to Video Service.
4. Add Next.js page and polling flow.
5. Scaffold Video Service repository or subfolder with FastAPI + Celery + Redis compose.
6. Port reusable VoiceFrame modules (Gemini/invoke/voice/video assembly) with config abstraction.

This gives you the cleanest path to add prompt-driven animated video generation without disrupting your current module architecture.